

Structures, Classes and Object-Oriented Foundations in C++

📌 Definition to Remember

A struct in C++ is a user-defined type identical to a class except that its members and base classes are public by default. Structures encapsulate data attributes and member functions.

1. Concept Overview & Fundamentals 🧠

Structures, Classes and Object-Oriented Foundations in C++ provides modern high-performance capabilities combining low-level hardware control with high-level abstractions, object-oriented paradigms, and Standard Template Library (STL) utilities.

In modern C++ (C++11 through C++20):

- **Zero-Overhead Principle:** What you don't use, you don't pay for; what you do use, you couldn't hand code any better.
- **Type Safety & Streams:** Type-safe stream I/O (`std::cin` , `std::cout`) prevents runtime format mismatch vulnerabilities.
- **Resource Acquisition Is Initialization (RAII):** Automatic resource management via constructors, destructors, and smart pointers.

2. Technical Architecture & Memory Mechanics ⚙️

C++ Runtime Execution Architecture

```
+-----+
|                STACK SEGMENT                |
| - Local scope objects, references, RAII stack frames |
| - Deterministic automatic destruction upon scope exit |
+-----+
|
| v
| ^
|
+-----+
|                HEAP / FREE STORE              |
| - Dynamic objects allocated via new, std::vector, etc. |
| - Managed via delete / smart pointers          |
+-----+
|                STATIC & GLOBAL MEMORY         |
| - Global objects, static class variables, string literals |
+-----+
|                CODE / TEXT SEGMENT            |
| - Executable machine code, inlined functions, templates |
+-----+
```

Modern C++ Feature Comparison

Modern C++ Paradigm	Legacy Equivalent	Engineering Advantage	Performance Impact
<code>std::cin</code> / <code>std::cout</code>	<code>scanf</code> / <code>printf</code>	Type-safe, extensible for custom classes	Buffered I/O (optimizable via <code>ios::sync_with_stdio(0)</code>)
References (<code>Type&</code>)	Pointers (<code>Type*</code>)	Non-nullable, clean syntax without dereferencing	Direct alias; zero runtime overhead
<code>std::vector<T></code>	Raw Dynamic Array (<code>malloc</code>)	Automatic resizing, bounds safety, memory management	Amortized O(1) push_back, contiguous cache
<code>nullptr</code>	<code>NULL</code> / <code>0</code>	Strongly typed pointer literal preventing integer overload bugs	Zero overhead

3. Core Syntax & Implementation Patterns

Modern C++ Template Structure

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>

// Fast I/O and standard namespace configuration
int main() {
    std::ios_base::sync_with_stdio(false);
    std::cin.tie(NULL);

    std::cout << "Executing C++ Module: Structures" << "\n";

    return 0;
}
```

4. Real-World Applications & System Engineering

- **Quantitative Finance & HFT:** High-frequency trading platforms use modern C++ for microsecond-level execution.
- **Game Engine Architecture:** Unreal Engine, Unity Core, and AAA game engines utilize C++ for real-time 3D rendering.
- **Deep Learning Frameworks:** Core tensor compute backends (PyTorch, TensorFlow, ONNX Runtime) are written in C++ and CUDA.
- **Web Browsers & Compilers:** Google Chrome (V8 engine), LLVM/Clang, and GCC are implemented in C++.

5. Comprehensive Solved Problems & Code Walkthroughs

Problem 1: book array initializer

Problem Description & Objective:

Implementation of book array initializer

C++ Code Implementation:

```

/*
 * Question:
 * Initialize an array of Book structures with different data for each book using initializer lists.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Struct Definition & Vector Aggregation (`std::vector<Book>`).
 *
 * 2. WHY IT IS USED?
 *    - Combines user-defined data structures with container storage.
 *
 * 3. HOW IT IS USED?
 *    - `vector<Book> library = { {"Title", "Author", 49.99}, ... };`
 *
 * 4. ALGORITHM EXPLANATION:
 * *   Step 1: Define `struct Book { string title; string author; double price; };`.
 * *   Step 2: Initialize `vector<Book> library`.
 * *   Step 3: Range loop displays catalog.
 * =====
 */

#include <iostream>
#include <string>
#include <vector>

// Use standard library namespace globally
using namespace std;

struct Book {
    string title;
    string author;
    double price;
};

// --- Main Execution Entry Point ---
int main() {
    // Declare dynamic C++ std::vector container
    vector<Book> library = {
        {"C++ Primer", "Stanley Lippman", 49.99},
        {"The C++ Programming Language", "Bjarne Stroustrup", 59.99},
        {"Effective Modern C++", "Scott Meyers", 44.95}
    };

    // Output formatted data to console stream
    cout << "--- Library Book Catalog ---" << endl;
    // Loop execution block
    for (size_t i = 0; i < library.size(); i++) {
        // Output formatted data to console stream
        cout << "Book " << (i + 1) << ": " << library[i].title
            << " by " << library[i].author
            << " ($" << library[i].price << ")" << endl;
    }
}

```

```
// Return zero code indicating successful program finish  
return 0;  
}
```

Problem 2: book structure

Problem Description & Objective:

Implementation of book structure

C++ Code Implementation:

```

/*
 * Question:
 * Create a program where you need to store and process data for a Book with attributes like title,
 * author, and price, demonstrating why a structure is more suitable than separate variables.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Encapsulation with User-Defined `struct`.
 *
 * 2. WHY IT IS USED?
 *    - Groups heterogeneous attributes (text + numbers) into a single entity.
 *
 * 3. HOW IT IS USED?
 *    - `struct Book { string title; string author; double price; };`
 *
 * 4. ALGORITHM EXPLANATION:
 * *   Step 1: Define `struct Book`.
 *   Step 2: Create instance `b1` and assign field values.
 *   Step 3: Pass `const Book &b` to `displayBook` function to print details.
 * =====
 */

#include <iostream>
#include <string>

// Use standard library namespace globally
using namespace std;

struct Book {
    string title;
    string author;
    double price;
};

void displayBook(const Book &b) {
    // Output formatted data to console stream
    cout << "Title: " << b.title << endl;
    // Output formatted data to console stream
    cout << "Author: " << b.author << endl;
    // Output formatted data to console stream
    cout << "Price: $" << b.price << endl;
}

// --- Main Execution Entry Point ---
int main() {
    Book b1;
    b1.title = "Clean Code";
    b1.author = "Robert C. Martin";
    b1.price = 35.50;

    // Output formatted data to console stream
    cout << "Book Details:" << endl;
    displayBook(b1);
}

```

```
// Return zero code indicating successful program finish
return 0;
}
```

Problem 3: car description function

Problem Description & Objective:

Implementation of car description function

C++ Code Implementation:

```

/*
 * Question:
 * Pass a Car structure to a function that prints out a description of the car in one complete sentence.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Const Reference Parameter Passing (`const StructType &`).
 *
 * 2. WHY IT IS USED?
 *    - Avoids copying structure data while ensuring function cannot modify original struct.
 *
 * 3. HOW IT IS USED?
 *    - `void printCarDescription(const Car &c)`
 *
 * 4. ALGORITHM EXPLANATION:
 *    * Step 1: Define `Car` struct.
 *    * Step 2: Create `myCar`.
 *    * Step 3: Pass `myCar` by const reference.
 *    * Step 4: Function formats and prints sentence.
 * =====
 */

#include <iostream>
#include <string>

// Use standard library namespace globally
using namespace std;

struct Car {
    string make;
    string model;
    int year;
    string color;
};

void printCarDescription(const Car &c) {
    // Output formatted data to console stream
    cout << "This car is a " << c.year << " " << c.color << " "
         << c.make << " " << c.model << "." << endl;
}

// --- Main Execution Entry Point ---
int main() {
    Car myCar = {"Toyota", "Corolla", 2023, "Blue"};

    printCarDescription(myCar);

    // Return zero code indicating successful program finish
    return 0;
}

```

Problem 4: car structure

Problem Description & Objective:

Implementation of car structure

C++ Code Implementation:

```

/*
 * Question:
 * Define a Car structure with fields for make, model, year, and color.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Struct Field Member Access (`car.field`).
 *
 * 2. WHY IT IS USED?
 *    - Defines a structured type representing physical objects.
 *
 * 3. HOW IT IS USED?
 *    - `car1.make = "Tesla";`
 *
 * 4. ALGORITHM EXPLANATION:
 * *   Step 1: Declare `struct Car`.
 *   Step 2: Instantiate `car1`.
 *   Step 3: Assign attributes.
 *   Step 4: Output specs via member access operator `.`.
 * =====
 */

#include <iostream>
#include <string>

// Use standard library namespace globally
using namespace std;

struct Car {
    string make;
    string model;
    int year;
    string color;
};

// --- Main Execution Entry Point ---
int main() {
    Car car1;
    car1.make = "Tesla";
    car1.model = "Model 3";
    car1.year = 2024;
    car1.color = "Red";

    // Output formatted data to console stream
    cout << "Car Specs:" << endl;
    // Output formatted data to console stream
    cout << "Make: " << car1.make << "\nModel: " << car1.model
         << "\nYear: " << car1.year << "\nColor: " << car1.color << endl;

    // Return zero code indicating successful program finish
    return 0;
}

```

Problem 5: nested structure student books

Problem Description & Objective:

Implementation of nested structure student books

C++ Code Implementation:

```

/*
 * Question:
 * Write a function where the Student structure also has books they have borrowed inside, showing nested
structure usage.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Nested Data Structures (`vector<Book>` inside `Student`).
 *
 * 2. WHY IT IS USED?
 *    - Models complex hierarchical real-world data (1-to-many relationships).
 *
 * 3. HOW IT IS USED?
 *    - `struct Student { int id; string name; vector<Book> borrowedBooks; };`
 *
 * 4. ALGORITHM EXPLANATION:
 * *   Step 1: Define `struct Book` and `struct Student`.
 *   Step 2: Add `vector<Book>` inside `Student`.
 *   Step 3: Populate student with multiple borrowed books.
 *   Step 4: Display nested structure content.
 * =====
 */

#include <iostream>
#include <string>
#include <vector>

// Use standard library namespace globally
using namespace std;

struct Book {
    string title;
    string author;
};

struct Student {
    int id;
    string name;
    // Declare dynamic C++ std::vector container
    vector<Book> borrowedBooks;
};

void displayStudentInfo(const Student &s) {
    // Output formatted data to console stream
    cout << "Student ID: " << s.id << ", Name: " << s.name << endl;
    // Output formatted data to console stream
    cout << "Borrowed Books:" << endl;
    // Loop execution block
    for (const auto &book : s.borrowedBooks) {
        // Output formatted data to console stream
        cout << "    - " << book.title << " by " << book.author << endl;
    }
}

```

```
// --- Main Execution Entry Point ---
int main() {
    Student student1;
    student1.id = 101;
    student1.name = "Alice";
    student1.borrowedBooks = {
        {"Design Patterns", "Erich Gamma"},
        {"Data Structures & Algorithms", "Mark Allen Weiss"}
    };

    displayStudentInfo(student1);

    // Return zero code indicating successful program finish
    return 0;
}
```

Problem 6: student modify gpa

Problem Description & Objective:

Implementation of student modify gpa

C++ Code Implementation:

```

/*
 * Question:
 * Write a function that accepts a pointer to a Student structure with fields for id, name, year, gpa and
 * modifies its grades.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Structure Pointers & Arrow Member Access Operator (`ptr->member`).
 *
 * 2. WHY IT IS USED?
 *    - Passing structure pointer allows function to modify original struct members directly.
 *
 * 3. HOW IT IS USED?
 *    - `void updateGPA(Student *s, double newGPA) { s->gpa = newGPA; }`
 *
 * 4. ALGORITHM EXPLANATION:
 * *   Step 1: Define `struct Student`.
 * *   Step 2: Create instance `s1`.
 * *   Step 3: Call `updateGPA(&s1, 3.9)`.
 * *   Step 4: `s->gpa` updates original `s1.gpa`. Output new GPA.
 * =====
 */

#include <iostream>
#include <string>

// Use standard library namespace globally
using namespace std;

struct Student {
    int id;
    string name;
    int year;
    double gpa;
};

void updateGPA(Student *s, double newGPA) {
    // Evaluate conditional decision logic
    if (s != nullptr) {
        s->gpa = newGPA;
    }
}

// --- Main Execution Entry Point ---
int main() {
    Student s1 = {1001, "Bob", 2, 3.4};

    // Output formatted data to console stream
    cout << "Before GPA update: " << s1.name << "'s GPA = " << s1.gpa << endl;

    updateGPA(&s1, 3.9);

    // Output formatted data to console stream

```

```
cout << "After GPA update:  " << s1.name << "'s GPA = " << s1.gpa << endl;

// Return zero code indicating successful program finish
return 0;

}
```

6. Exam Scoring Points & Memory Revision

★ Must-Write Points

1. struct vs class in C++: struct defaults to public access; class defaults to private access.
2. Structures in C++ can have constructors, destructors, and member methods.
3. Array of structures: Efficiently stores and processes database records (e.g. Students, Employees).
4. Passing structs: Prefer pass-by-const-reference (const StructType&) to avoid full object copying.
5. Memory layout: Total size = sum of member sizes + alignment padding bytes.

⚡ Quick Recall

Encapsulated Attributes -> Member Functions -> Constructors -> public default -> const Ref Passing